

MD5 Collision Attack Lab

姓名：王子卓

学号：57123242

实验报告需求：按实验指导书按步执行，实验报告记录执行的命令，对执行结果截屏，并对指导书中提出的问题进行回答和截屏展示。

1、实验环境设置

本实验在 SEED 环境中进行，不使用容器。

2、实验步骤

2.1 Task 1: Generating Two Different Files with the Same MD5 Hash

理论上哈希函数具有强、弱抗碰撞性，但只是“很难”找出，而不是不可能。对于 MD5，它的抗碰撞性已经被攻破了。2004 年，王小云教授等人在 CRYPTO 会议上展示了 MD5 的碰撞攻击。后来 Marc Stevens 等人进一步优化，做出了可以实际运行的工具，也就是本次实验里用的 md5collgen，它是专门用来生成 MD5 碰撞文件的工具。但它不是“随便输入两个文件，就立刻让它们 MD5 相同”。它做的事情有严格限制：

我们先给它一个 prefix，也就是两个文件共同的开头；Md5collgen 它会在 prefix 后面生成两个不同的 128 字节碰撞块，P 和 Q；最终输出两个文件：

文件 1: prefix + padding + P；文件 2: prefix + padding + Q

这两个文件整体 MD5 相同，但 P 和 Q 不完全一样。

所以，它不是“破解任意 MD5”，而是利用 MD5 的已知缺陷，构造出一对碰撞。这个构造过程需要特定算法和计算，不是随便按一下按钮就出来的。只是因为这个工具已经高度优化，所以在普通电脑上也能较快生成。

MD5 的内部结构是：把输入按 64 字节分块，每块经过一个压缩函数，产生 128 位中间哈希值，最后得到 128 位 MD5 值。因为 MD5 的压缩函数存在差分路径缺陷。攻击者可以精心构造两个不同的 64 字节块，使得它们经过压缩函数后，输出的中间哈希值相同。然后，只要它们前面的 prefix 相同，后面的 suffix 也相同，最终整体哈希就会相同。

md5collgen 做的就是：接收我们的 prefix，自动补 padding，让 prefix 对齐到 64 字节边界生成一对 128 字节的碰撞块 P 和 Q，最后输出两个文件。

任务：根据任务书要求使用 md5collgen 软件生成具有相同 HASH 的两个二进制文件。并

使用 diff 和 md5sum 比较这两个文件

位置: SEED

命令:

```
echo "This is a common prefix for MD5 collision lab." > prefix.txt
md5collgen -p prefix.txt -o out1.bin out2.bin
diff out1.bin out2.bin
md5sum out1.bin out2.bin
ls -l out1.bin out2.bin
cmp -l out1.bin out2.bin
```

截图

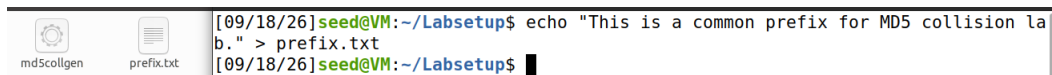
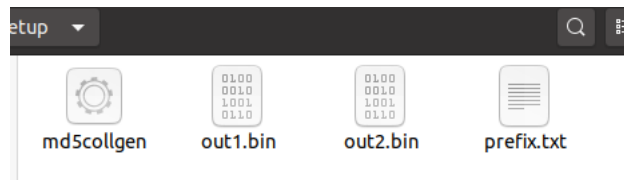


图 1: 创建一个普通 prefix



```
[09/18/26]seed@VM:~/Labsetup$ md5collgen -p prefix.txt -o out1.bin out2.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'out1.bin' and 'out2.bin'
Using prefixfile: 'prefix.txt'
Using initial value: 09d233bdac16de446d8063377f2e7c28

Generating first block: .....
Generating second block: W.
Running time: 9.17492 s
[09/18/26]seed@VM:~/Labsetup$
```

图 2: 生成两个碰撞文件

```
[09/18/26]seed@VM:~/Labsetup$ diff out1.bin out2.bin
Binary files out1.bin and out2.bin differ
```

图 3: 查看文件是否不同

```
[09/18/26]seed@VM:~/Labsetup$ md5sum out1.bin out2.bin
216712fa44924ff8bea5a64d427d0b9c out1.bin
216712fa44924ff8bea5a64d427d0b9c out2.bin
```

图 4: 查看 MD5 是否相同

```
[09/18/26]seed@VM:~/Labsetup$ ls -l out1.bin out2.bin
-rw-rw-r-- 1 seed seed 192 Sep 18 01:08 out1.bin
-rw-rw-r-- 1 seed seed 192 Sep 18 01:08 out2.bin
```

图 5: 查看文件长度

```
[09/18/26]seed@VM:~/Labsetup$ cmp -l out1.bin out2.bin
84 52 252
110 333 133
111 10 11
124 122 322
148 105 305
174 174 374
175 266 265
188 337 137
```

图 6: 查看具体哪些字节不同, 差异字节都在碰撞块中

2.1.2 问题 1:: If the length of your prefix file is not multiple of 64, what is going to happen?

prefix+padding+128 字节碰撞块, md5collgen 会自动对 prefix 做对齐处理, 添加 padding, 使实际参与碰撞生成的前缀部分对齐到 64 字节边界。因此输出文件开头是: 两个输出文件的 prefix 和 padding 相同, 差异只在后面的 128 字节碰撞块。文件长度会比原来的 prefix 更长。

2.1.3 问题 2: Create a prefix file with exactly 64 bytes, and run the collision tool again, and see what happens.

命令:

```
python3 -c 'import sys; sys.stdout.buffer.write(b"A"*64)' > prefix64.txt
wc -c prefix64.txt
md5collgen -p prefix64.txt -o out64_1.bin out64_2.bin
ls -l out64_1.bin out64_2.bin
md5sum out64_1.bin out64_2.bin
cmp -l out64_1.bin out64_2.bin | head
```

观察:

```
[09/18/26]seed@VM:~/Labsetup$ python3 -c 'import sys; sys.stdout.buffer.write(b"
A"*64)' > prefix64.txt
[09/18/26]seed@VM:~/Labsetup$ wc -c prefix64.txt
64 prefix64.txt
[09/18/26]seed@VM:~/Labsetup$ md5collgen -p prefix64.txt -o out64_1.bin out64_2.
bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'out64_1.bin' and 'out64_2.bin'
Using prefixfile: 'prefix64.txt'
Using initial value: b217e7185a63fe5f643fe6a6d401bf59

Generating first block: ....
Generating second block: S00.....
Running time: 8.80986 s
```

因为 prefix 已经是 64 字节的倍数, md5collgen 不需要再补对齐 padding, 直接将它作为第一个 MD5 块, 所以输出文件长度是: 64+128=192 字节, 两个文件仍然 MD5 相同, 差异仍然只在 128 字节碰撞块里。

```
cmp -n 64 prefix64.txt out64_1.bin
```

```
[09/18/26]seed@VM:~/Labsetup$ cmp -n 64 prefix64.txt out64_1.bin
[09/18/26]seed@VM:~/Labsetup$ █
```

2.1.4 问题 3: Are the data (128 bytes) generated by md5collgen completely different for the two output files? Please identify all the bytes that are different.

不是完全不同。两个 128 字节碰撞块大部分字节相同，只有少数几个字节不同。

```
cmp -l out64_1.bin out64_2.bin

[09/18/26]seed@VM:~/Labsetup$ ls -l out64_1.bin out64_2.bin
-rw-rw-r-- 1 seed seed 192 Sep 18 02:01 out64_1.bin
-rw-rw-r-- 1 seed seed 192 Sep 18 02:01 out64_2.bin
[09/18/26]seed@VM:~/Labsetup$ md5sum out64_1.bin out64_2.bin
105396882f1521c74ff4d9319628c6de out64_1.bin
105396882f1521c74ff4d9319628c6de out64_2.bin
[09/18/26]seed@VM:~/Labsetup$ cmp -l out64_1.bin out64_2.bin | head
 84  67 267
110  16 216
124 172 372
148  47 247
174 372 172
188 112 312
```

由 `cmp -l` 可见，两个文件的差异不是 128 字节全部不同，而是只有少数几个字节不同，具体偏移和值如截图所示。

2.2 Task 2: Understanding MD5's Property

任务：MD5 算法的一个特征如果是两个文件具有相同的 hash，那么为他们加上相同的后缀，他们的 hash 仍然相同，请针对这个特征设计一个实验，展现 MD5 算法的这个特征。

环境：SEED

实验设计：

验证如果两个文件 M 和 N 的 MD5 相同，即 $MD5(M) = MD5(N)$ ；那么给它们加上同一个后缀 T 后，新的两个文件仍然 MD5 相同，即 $MD5(M|T) = MD5(N|T)$

命令：

```
echo "This is a common suffix for MD5 collision lab." > suffix.txt
cat suffix.txt
ls -l suffix.txt
cat out1.bin suffix.txt > out1_suffix.bin
cat out2.bin suffix.txt > out2_suffix.bin
ls -l out1_suffix.bin out2_suffix.bin
md5sum out1_suffix.bin out2_suffix.bin
cmp -l out1_suffix.bin out2_suffix.bin | head
```

截图（可以有多个）

```
[09/18/26]seed@VM:~/Labsetup$ echo "This is a common suffix for MD5 collision la
b." > suffix.txt
[09/18/26]seed@VM:~/Labsetup$ cat suffix.txt
This is a common suffix for MD5 collision lab.
[09/18/26]seed@VM:~/Labsetup$ ls -l suffix.txt
-rw-rw-r-- 1 seed seed 47 Sep 18 02:31 suffix.txt
[09/18/26]seed@VM:~/Labsetup$ cat out1.bin suffix.txt > out1_suffix.bin
[09/18/26]seed@VM:~/Labsetup$ cat out2.bin suffix.txt > out2_suffix.bin
[09/18/26]seed@VM:~/Labsetup$ ls -l out1_suffix.bin out2_suffix.bin
-rw-rw-r-- 1 seed seed 239 Sep 18 02:31 out1_suffix.bin
-rw-rw-r-- 1 seed seed 239 Sep 18 02:32 out2_suffix.bin
[09/18/26]seed@VM:~/Labsetup$ md5sum out1_suffix.bin out2_suffix.bin
a37c06acf5f6a2a3f8be50a32cdb3072 out1_suffix.bin
a37c06acf5f6a2a3f8be50a32cdb3072 out2_suffix.bin
```

图 1：两个输出文件的哈希值是相同的

<pre>[09/18/26]seed@VM:~/Labsetup\$ cmp -l out1_suffix.bin out2_suffix.bin head 84 52 252 110 333 133 111 10 11 124 122 322 148 105 305 174 174 374 175 266 265 188 337 137</pre>
图 2：对比查看发现文件是不同的
结果说明： 拼接相同后缀后，两个新文件的 MD5 仍然完全相同。这说明 MD5 的碰撞性质可以向后传递：只要两个文件在某一点具有相同的中间哈希值，那么之后添加相同的数据不会改变最终哈希的相同性。

2.3 Task 3: Generating Two Executable Files with the Same MD5 Hash

任务：按任务书要求，创建两个 HASH 值相同的二进制程序，他们的 XYZ 数组存储了不同的内容。 环境：SEED
<pre>命令 gcc -o task3 task3.c ./task3 xxd task3 > task3.hex grep -n "4141" task3.hex head head -c 12288 task3 > prefix tail -c +12417 task3 > suffix ls -l task3 prefix suffix md5collgen -p prefix -o out_p out_q tail -c 128 out_p > P tail -c 128 out_q > Q ls -l P Q cat prefix P suffix > task3_p cat prefix Q suffix > task3_q chmod +x task3_p task3_q md5sum task3_p task3_q ./task3_p ./task3_q</pre>
截图（可以有多组）


```

./task4
xxd task4 | grep -n "4141 4141 4141 4141" | head -n 20

head -c 12288 task4 > prefix
tail -c +12545 task4 > suffix
ls -l task4 prefix suffix

md5collgen -p prefix -o out_p out_q
tail -c 128 out_p > P
tail -c 128 out_q > Q
ls -l P Q

cat prefix P P suffix > task4_benign
cat prefix Q P suffix > task4_malicious
chmod +x task4_benign task4_malicious

md5sum task4_benign task4_malicious
./task4_benign
./task4_malicious

```

截图（可以有多组）



```

seed@VM: ~/Labsetup
GNU nano 4.8 task4.c Modified
#include <stdio.h>
#include <string.h>

volatile unsigned char X[128];
volatile unsigned char Y[128];

int main()
{
    int i;
    for (i = 0; i < 128; i++) {
        X[i] = 0x41;
        Y[i] = 0x41;
    }

    if (memcmp((const void*)X, (const void*)Y, 128) == 0) {
        printf("Benign code executed!\n");
    } else {
        printf("Malicious code executed!\n");
    }
}

^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^\ Replace  ^U Paste Text ^T To Spell  ^_ Go To Line

```

图 1: 编译所需 C 文件

